Contents

Chapter 3

Linear systems

3.1 Sensitivity analysis

3.1.1 Vector and matrix norms

To describe the effect of rounding errors on algorithms in linear algebra, we need the following concept.

Definition 1 *A norm on $\mathbb{R}^n$ is a function that assigns to each $\boldsymbol{v} \in \mathbb{R}^n$ a real number $\|\boldsymbol{v}\|$ in such a way that, for all vectors $\boldsymbol{v}, \boldsymbol{w} \in \mathbb{R}^n$ and all scalars $\alpha \in \mathbb{R}$,*

1. $\|\boldsymbol{v}\| \geq 0$;
2. $\|\boldsymbol{v}\| = 0$ *iff* $\boldsymbol{v} = 0$;
3. $\|\alpha\boldsymbol{v}\| = |\alpha|\|\boldsymbol{v}\|$;
4. $\|\boldsymbol{v} + \boldsymbol{w}\| \leq \|\boldsymbol{v}\| + \|\boldsymbol{w}\|$ *(triangle inequality).*

The p-norm of $\boldsymbol{v} = (v_1, v_2, \ldots, v_n) \in \mathbb{R}^n$ is defined by

$$\|\boldsymbol{v}\|_p = \left(\sum_{j=1}^{n} |v_j|^p \right)^{1/p} \qquad \text{for } 1 \leq p < \infty,$$

and the ∞-norm by

$$\|\boldsymbol{v}\|_\infty = \max_{1 \leq j \leq n} |v_j| \text{ for all } \boldsymbol{v} \in \mathbb{R}^n.$$

Properties 1-3 in the definition are obvious, but 4 is more difficult.

Notice that the 2-norm is the same as the Euclidean norm,

$$\|\boldsymbol{v}\|_2 = \left(\sum_{j=1}^{n} v_j^2 \right)^{1/2}$$

Definition 2 (Induced matrix norm) *Given a norm $\|\cdot\|$ on $\mathbb{R}^n$, the induced matrix norm is defined by*

$$\|A\| = \max_{\|\boldsymbol{w}\|=1} \|A\boldsymbol{w}\| = \max_{\boldsymbol{v}\neq\boldsymbol{0}} \frac{\|A\boldsymbol{v}\|}{\|\boldsymbol{v}\|} \quad \text{for} \quad A \in \mathbb{R}^{n\times n}.$$

Theorem 3.1.1 *The induced matrix norm $\|A\|$ is the smallest constant C such that*

$$\|A\boldsymbol{v}\| \leq C\|\boldsymbol{v}\| \quad \text{for all} \quad \boldsymbol{v} \in \mathbb{R}^n.$$

Proof. Let C be a constant such that $\|A\boldsymbol{v}\| \leq C\|\boldsymbol{v}\|$ for all vectors $\boldsymbol{v} \in \mathbb{R}^n$. For $\boldsymbol{v} \neq \boldsymbol{0}$, we have

$$\frac{\|A\boldsymbol{v}\|}{\|\boldsymbol{v}\|} \leq C.$$

Taking the maximum over all vectors $\boldsymbol{v} \neq \boldsymbol{0}$, we have

$$\max_{\boldsymbol{v}\neq\boldsymbol{0}} \frac{\|A\boldsymbol{v}\|}{\|\boldsymbol{v}\|} \leq C,$$

or equivalently,

$$\|A\| \leq C.$$

Hence $\|A\|$ is the smallest constant C such that $\|A\boldsymbol{v}\| \leq C\|\boldsymbol{v}\|$ for all vectors $\boldsymbol{v} \in \mathbb{R}^n$. □

Theorem 3.1.2 *Any induced matrix norm on $\mathbb{R}^{n\times n}$ has the properties*

1. $\|A\| \geq 0$;
2. $\|A\| = 0$ *iff* $A = 0$;
3. $\|\alpha A\| = |\alpha|\|A\|$;
4. $\|A + B\| \leq \|A\| + \|B\|$.

We denote by $\|A\|_p$ the matrix norm induced by the p-norm on $\mathbb{R}^n$. In practice, $\|A\|_p$ is difficult to compute unless $p = 1$ or ∞.

Theorem 3.1.3 *For $A = [a_{ij}] \in \mathbb{R}^{n\times n}$,*

$$\|A\|_1 = \max_{1\leq j\leq n} \sum_{i=1}^{n} |a_{ij}| = \text{ maximum absolute column sum,}$$

$$\|A\|_\infty = \max_{1\leq i\leq n} \sum_{j=1}^{n} |a_{ij}| = \text{ maximum absolute row sum.}$$

Proof. From the definition,

$$\|A\|_1 = \max_{\boldsymbol{v}\neq 0} \frac{\|A\boldsymbol{v}\|_1}{\|\boldsymbol{v}\|_1}.$$

We have

$$\begin{aligned}\|A\boldsymbol{v}\|_1 &= \sum_{i=1}^{n}\left|\sum_{j=1}^{n} a_{ij}v_j\right| \leq \sum_{i=1}^{n}\sum_{j=1}^{n}|a_{ij}||v_j| \\ &= \sum_{j=1}^{n}\sum_{i=1}^{n}|a_{ij}||v_j| \leq \max_{1\leq j\leq n}\left(\sum_{i=1}^{n}|a_{ij}|\right)\sum_{j=1}^{n}|v_j|,\end{aligned}$$

which implies that (cf. Theorem 3.1.1)

$$\|A\|_1 \leq \max_{1\leq j\leq n}\left(\sum_{i=1}^{n}|a_{ij}|\right). \tag{3.1.1}$$

To show equality indeed can be achieved in (3.1.1), without loss of generality, we can assume that for some j_0

$$\sum_{i=1}^{n}|a_{ij_0}| = \max_{1\leq j\leq n}\left(\sum_{i=1}^{n}|a_{ij}|\right).$$

Choose $\boldsymbol{v}^*$ so that

$$v_j^* = \begin{cases}1 & \text{if } j = j_0, \\ 0 & \text{if } j \neq j_0.\end{cases}$$

Then $\|A\boldsymbol{v}^*\|_1 = \sum_{i=1}^{n}|a_{ij_0}| = \|A\|_1$ (since $\|\boldsymbol{v}^*\|_1 = 1$) and equality has achieved in (3.1.1).

Similar techniques can be used to prove the second statement of the theorem on the computation of $\|A\|_\infty$. □

Example 3.1.1 If

$$A = \begin{bmatrix} 1 & -3 & 4 \\ 2 & 0 & 1 \\ -6 & 8 & 1 \end{bmatrix}$$

then

$$\begin{aligned}\|A\|_1 &= \max(1+2+6, 3+0+8, 4+1+1) = 11, \\ \|A\|_\infty &= \max(1+3+4, 2+0+1, 6+8+1) = 15.\end{aligned}$$

The MATLAB function `norm(x,p)` can be used to compute norms.

MATLAB commands

```
>> v=[3,-1,4]
   v = 3 -1 4
>> norm(v,1)
   ans = 8
>> norm(v) % same as norm(v,2)
   ans = 5.09901951359278
>> norm(v,Inf)
   ans = 4
>> A=[1,-3,4;2,0,1;-6,8,1]
 A = 1 -3  4
     2  0  1
    -6  8  1
>> norm(A,1)
   ans = 11
>> norm(A) % same as norm(A,2)
   ans = 10.51490723603980
>> norm(A,inf)
   ans = 15
```

The function `linalg.norm(v,p)` from SciPy package can be used to compute norms.

Python

```
>>> import scipy
>>> import scipy.linalg
>>> v = scipy.array([3, -1, 4])
>>> scipy.linalg.norm(v,1)
8.0
>>> scipy.linalg.norm(v) # same as scipy.linalg.norm(v,2)
5.0990195135927845
>>> scipy.linalg.norm(v,scipy.inf)
4.0
>>> A = scipy.array([[1,-3,4],[2,0,1],[-6,8,1]])
>>> print(A)
[[ 1 -3  4]
 [ 2  0  1]
 [-6  8  1]]
>>> scipy.linalg.norm(A,1)
11.0
>>> scipy.linalg.norm(A) % same as scipy.linalg.norm(A,2)
11.489125293076057
>>> scipy.linalg.norm(A,scipy.inf)
15.0
```

3.1.2 Condition number

Definition 3 *Given a norm on $\mathbb{R}^n$, we define the condition number of the matrix A by*

$$\kappa(A) = \|A\|\|A^{-1}\|.$$

We define $\kappa(A) = \infty$ whenever A is singular.

Example 3.1.2 If

$$A = \begin{bmatrix} 1 & -1 \\ k & -1 \end{bmatrix}$$

for some $k > 1$ then

$$A^{-1} = \frac{1}{k-1}\begin{bmatrix} -1 & 1 \\ -k & 1 \end{bmatrix}$$

so that

$$\|A\|_1 = k+1 \quad \|A^{-1}\|_1 = \frac{k+1}{k-1} \quad \text{and } \kappa(A) = \frac{(k+1)^2}{(k-1)}.$$

If $k = 1+\delta$ with some small δ, $\kappa(A) = (2+\delta)^2/\delta$ which can become very large.

Definition 4 *(Ill-conditioned matrices) If $\kappa(A)$ is large, then A is said to be an ill-conditioned matrix.*

Ill-conditioness of a matrix is a norm-dependent property. However, any two condition numbers are equivalent.

Lemma 3.1.4 *If $A \in \mathbb{R}^{n\times n}$ then*

$$\begin{aligned}
\frac{1}{n}\kappa_2(A) \le \kappa_1(A) \le n\kappa_2(A)\\
\frac{1}{n}\kappa_\infty(A) \le \kappa_2(A) \le n\kappa_\infty(A)\\
\frac{1}{n^2}\kappa_1(A) \le \kappa_\infty(A) \le n^2\kappa_1(A).
\end{aligned}$$

Proof. See Tutorial Weeks 3/4, Problem 2. □

Theorem 3.1.5 *Let $A, B \in \mathbb{R}^{n\times n}$ and let $I = [\delta_{ij}]$ denote the identity matrix in $\mathbb{R}^{n\times n}$. For any induced norm $\|\cdot\|$ on $R^{n\times n}$,*

1. *$\|AB\| \le \|A\|\|B\|$ and $\kappa(AB) \le \kappa(A)\kappa(B)$,*
2. *$\|I\| = 1$ and $\kappa(I) = 1$,*
3. *$\kappa(\mu A) = \kappa(A)$ for any scalar $\mu \neq 0$,*
4. *$\kappa(A^{-1}) = \kappa(A)$,*
5. *$\kappa(A) \ge 1$.*

Proof. See Tutorial Weeks 3/4, Problem 3. □

Condition number κ_2 of a real symmetric matrix

Definition 5 *(Eigenvalues) Let $A \in \mathbb{R}^{n\times n}$ be an $n \times n$ matrix. A number $\lambda \in \mathbb{C}$ is called an eigenvalue of A if there exists a nonzero vector $\boldsymbol{v} \in \mathbb{C}^n$ such that*

$$A\boldsymbol{v} = \lambda\boldsymbol{v}$$

Theorem 3.1.6 *If $\lambda_1, \ldots, \lambda_n$ are the eigenvalues of an $n \times n$ real and symmetric matrix A such that*

$$|\lambda_1| \ge \ldots \ge |\lambda_n| > 0$$

then

$$\kappa_2(A) = \frac{|\lambda_1|}{|\lambda_n|}.$$

Proof. Since A is symmetric, there is a diagonal matrix D and an orthogonal matrix P so that $A = P^T DP$ and $PP^T = I$. We have

$$\begin{aligned}\|A\boldsymbol{v}\|_2^2 &= \boldsymbol{v}^T A^T A\boldsymbol{v} \\ &= \boldsymbol{v}^T P^T DPP^T DP\boldsymbol{v} = \boldsymbol{w}^T D^2 \boldsymbol{w} \text{ with } \boldsymbol{w} := P\boldsymbol{v} \\ &\leq |\lambda_1|^2 \|\boldsymbol{w}\|_2^2 = |\lambda_1|^2 \|\boldsymbol{v}\|_2^2,\end{aligned}$$

By choosing $\boldsymbol{w} = \boldsymbol{e}_1$ (the first unit vector), we see that $\|A\|_2 = |\lambda_1|$.

Using similar techniques with $A^{-1} = P^T D^{-1} P$, we can show that $\|A^{-1}\|_2 = 1/|\lambda_n|$. Hence

$$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{|\lambda_1|}{|\lambda_n|}.$$

□

3.1.3 Stability

Let $A \in \mathbb{R}^{n\times n}$ be a non-singular matrix and consider the linear system

$$A\boldsymbol{x} = \boldsymbol{b}.$$

We now discuss the sensitivity of the solution $\boldsymbol{x}$ to small perturbations in A and $\boldsymbol{b}$. Let $\delta\boldsymbol{x}$ denote the change in $\boldsymbol{x}$ when A and $\boldsymbol{b}$ are changed by δA and $\delta\boldsymbol{b}$, respectively, so that

$$(A + \delta A)(\boldsymbol{x} + \delta\boldsymbol{x}) = \boldsymbol{b} + \delta\boldsymbol{b}.$$

The following theorem shows that if A is ill-conditioned then the change in $\boldsymbol{x}$, namely $\delta\boldsymbol{x}$, may be large even though the perturbations δA and $\delta\boldsymbol{b}$ are small.

Theorem 3.1.7 *Suppose that*

$$\frac{\|\delta A\|_p}{\|A\|_p} \leq \gamma < \frac{1}{\kappa_p(A)}.$$

Then $A + \delta A$ is invertible. Moreover, if $A\boldsymbol{x} = \boldsymbol{b}$ and $(A + \delta A)(\boldsymbol{x} + \delta\boldsymbol{x}) = \boldsymbol{b} + \delta\boldsymbol{b}$ then

$$\frac{\|\delta\boldsymbol{x}\|_p}{\|\boldsymbol{x}\|_p} \leq \frac{\kappa_p(A)}{1 - \gamma\kappa_p(A)} \left(\frac{\|\delta A\|_p}{\|A\|_p} + \frac{\|\delta\boldsymbol{b}\|_p}{\|\boldsymbol{b}\|_p} \right).$$

Proof. See Problem 13, Tutorial Weeks 03-04. □

We have the following conclusions:

- Relative error in solution $\preceq \kappa(A)\times$relative error in data
- In IEEE double precision (ϵ of order 10^{-15}), if $\kappa(A)$ is of order 10^k then expect about $15 - k$ correct digits in $\boldsymbol{x}_*$.

Example 3.1.3 Let

$$A = \begin{bmatrix} 2 & 1 \\ 4 & 2+\mu \end{bmatrix} \text{ and } |\mu| < 1.$$

Show that

$$A^{-1} = \frac{1}{2\mu} \begin{bmatrix} 2+\mu & -1 \\ -4 & 2 \end{bmatrix}.$$

Find $\kappa_1(A)$. What is the limiting behaviour of $\kappa_1(A)$ when $\mu \to 0$?

It is easy to check that $AA^{-1} = A^{-1}A = I$.

We have

$$\|A\|_1 = \max(2+4, 1+2+|\mu|) = 6$$

and

$$\|A^{-1}\|_1 = \frac{1}{2|\mu|} \max(2+|\mu|+4, 1+2) = \frac{6+|\mu|}{2|\mu|}.$$

So

$$\kappa_1(A) = 6 \times \frac{6+|\mu|}{2|\mu|} = \frac{18+3|\mu|}{|\mu|}.$$

If $|\mu| \to 0$ then $\kappa_1(A) \to \infty$. Note that A tends to the singular matrix

$$\begin{bmatrix} 2 & 1 \\ 4 & 2 \end{bmatrix}$$

Consider the matrix in the previous example with $\mu = 0.1$. Solve $A\boldsymbol{x}_1 = \boldsymbol{b}$, $(A+\delta A)\boldsymbol{x}_2 = \boldsymbol{b}$, and $(A+\delta A)\boldsymbol{x}_3 = \boldsymbol{b} + \delta\boldsymbol{b}$ with

$$\boldsymbol{b} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \delta A = \begin{bmatrix} 0 & 0 \\ 0 & 0.1 \end{bmatrix}, \quad \delta\boldsymbol{b} = \begin{bmatrix} 0 \\ 0.1 \end{bmatrix}.$$

The equations to be solved are

$$\begin{bmatrix} 2 & 1 \\ 4 & 2.1 \end{bmatrix} \boldsymbol{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \begin{bmatrix} 2 & 1 \\ 4 & 2.2 \end{bmatrix} \boldsymbol{x}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \text{and} \quad \begin{bmatrix} 2 & 1 \\ 4 & 2.2 \end{bmatrix} \boldsymbol{x}_3 = \begin{bmatrix} 1 \\ 1.1 \end{bmatrix},$$

We have

$$\boldsymbol{x}_1 = \begin{bmatrix} 5.5 \\ -10 \end{bmatrix}, \quad \boldsymbol{x}_2 = \begin{bmatrix} 3 \\ -5 \end{bmatrix}, \boldsymbol{x}_3 = \begin{bmatrix} 2.75 \\ -4.5 \end{bmatrix},$$

so that

$$\frac{\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|_1}{\|\boldsymbol{x}_1\|_1} = \frac{7.5}{15.5} = 0.4839 \quad \text{and} \quad \frac{\|\boldsymbol{x}_1 - \boldsymbol{x}_3\|_1}{\|\boldsymbol{x}_1\|_1} = \frac{8.25}{15.5} = 0.5323.$$

On the other hand,

$$\frac{\|\delta A\|_1}{\|A\|_1} = \frac{0.1}{6} = 0.0167 \quad \text{and} \quad \frac{\|\delta\boldsymbol{b}\|_1}{\|\boldsymbol{b}\|_1} = \frac{0.1}{2} = 0.05.$$

Hence

$$\frac{\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|_1}{\|\boldsymbol{x}_1\|_1} \approx 29\frac{\|\delta A\|_1}{\|A\|_1} \quad \text{and} \quad \frac{\|\boldsymbol{x}_1 - \boldsymbol{x}_3\|_1}{\|\boldsymbol{x}_1\|_1} \approx 8\left(\frac{\|\delta A\|_1}{\|A\|_1} + \frac{\|\delta \boldsymbol{b}\|_1}{\|\boldsymbol{b}\|_1}\right)$$

This example supports Theorem 3.1.7 because $\kappa_1(A) = 183$ according to the previous example.

- Theorem 3.1.7 shows that when the matrix A is ill-conditioned, solving the linear system doesn't give accurate solutions.
- A remedy is to precondition the matrix A. Instead of solving $A\boldsymbol{x} = \boldsymbol{b}$ we solve $MA\boldsymbol{x} = M\boldsymbol{b}$ where M is an invertible matrix such that $\kappa(MA)$ is much smaller than $\kappa(A)$. We will not discuss this topic further.

3.2 LU factorisation

3.2.1 Solving a system by LU factorisation

Definition 6 *A matrix $L = [\ell_{ij}] \in \mathbb{R}^{n\times n}$ is said to be lower triangular if*

$$\ell_{ij} = 0 \quad \textit{for} \quad 1 \le i < j \le n.$$

We say that L is unit lower triangular if, in addition,

$$\ell_{ii} = 1 \quad \textit{for} \quad 1 \le i \le n.$$

Example 3.2.1 A 4×4 lower triangular matrix has the form

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ \ell_{21} & 1 & 0 & 0 \\ \ell_{31} & \ell_{32} & 1 & 0 \\ \ell_{41} & \ell_{42} & \ell_{43} & 1 \end{bmatrix}$$

A unit lower triangular system of equations is easily solved using forward elimination.

Example 3.2.2 To solve

$$\begin{array}{rrrrcr} x_1 & & & & = & 2, \\ -2x_1 & +x_2 & & & = & -5, \\ x_1 & +4x_2 & +x_3 & & = & -2, \\ 3x_1 & & +3x_3 & +x_4 & = & 10, \end{array}$$

we do

$$\begin{aligned} x_1 &= 2, \\ x_2 &= -5 + 2x_1 = -5 + 2 \times 2 = -1, \\ x_3 &= -2 - x_1 - 4x_2 = -2 - 2 - 4 \times (-1) = 0, \\ x_4 &= 10 - 3x_1 - 3x_3 = 10 - 3 \times 2 - 3 \times 0 = 4. \end{aligned}$$

Definition 7 *(Upper triangular matrices) A matrix $U = [u_{ij}] \in \mathbb{R}^{n\times n}$ is said to be upper triangular if*

$$u_{ij} = 0 \text{ for } 1 \le j < i \le n,$$

and unit upper triangular if, in addition, $u_{ii} = 1$ for all i.

Example 3.2.3 A 4×4 upper triangular matrix has the form

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{32} \\ 0 & 0 & 0 & u_{44} \end{bmatrix}.$$

An upper triangular linear system is easily solved using back substitution, provided every diagonal entry is non-zero, i.e., $u_{ii} \neq 0$ for $1 \le i \le n$.

Definition 8 *(LU-factorization) An LU-factorization of a matrix $A = [a_{ij}] \in \mathbb{R}^{n\times n}$ is a representation of the form*

$$A = LU,$$

for some unit lower triangular matrix L and some upper triangular matrix U.

If we know an LU-factorization of A then solving

$$A\boldsymbol{x} = \boldsymbol{b}$$

is easy:

- First solve $L\boldsymbol{y} = \boldsymbol{b}$ using forward elimination,
- Then solve $U\boldsymbol{x} = \boldsymbol{y}$ using back substitution.
- In this way,
$$A\boldsymbol{x} = (LU)\boldsymbol{x} = L(U\boldsymbol{x}) = L\boldsymbol{y} = \boldsymbol{b},$$
as required.

Example 3.2.4 Check that

$$\begin{bmatrix} 1 & 0 & 3 \\ 2 & 2 & 2 \\ 3 & 6 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 3 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 3 \\ 0 & 2 & -4 \\ 0 & 0 & 7 \end{bmatrix}$$

and then solve

$$\begin{bmatrix} 1 & 0 & 3 \\ 2 & 2 & 2 \\ 3 & 6 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \\ 7 \end{bmatrix}$$

To find the LU factorization we use the Gaussian elimination process.

1. Set $a_{ij}^{(1)} = a_{ij}$ for $i, j = 1, \ldots, n$;
2. For $k = 1, \ldots, n-1$ do

$$\begin{array}{l} \text{for } i = k+1, \ldots, n \\ \quad \ell_{ik} = \dfrac{a_{ik}^{(k)}}{a_{kk}^{(k)}} \\ \quad \text{for } j = k+1, \ldots, n \\ \quad\quad a_{ij}^{(k+1)} = a_{ij}^{(k)} - \ell_{ik} a_{kj}^{(k)} \\ \quad \text{end do} \\ \text{end do} \end{array}$$

3. At the end of the procedure, the entries of L are known, whereas the entries of U are

$$u_{ij} = a_{ij}^{(i)} \quad \text{for} \quad i = 1, \ldots, n \quad \text{and} \quad j = i, \ldots, n.$$

The elements $a_{kk}^{(k)}$ must all be different from zero and are called pivot elements.

Example 3.2.5 Find the LU-factorization of

$$A = \begin{bmatrix} 1 & 0 & 3 \\ 2 & 2 & 2 \\ 3 & 6 & 4 \end{bmatrix}$$

Applying the following row operations

$$\begin{array}{l} R_2 \leftarrow R_2 - 2R_1 \quad (\ell_{21} = 2) \\ R_3 \leftarrow R_3 - 3R_1 \quad (\ell_{31} = 3) \end{array} \quad \text{to obtain} \quad \begin{bmatrix} 1 & 0 & 3 \\ 0 & 2 & -4 \\ 0 & 6 & -5 \end{bmatrix}.$$

Now the following row operation

$$R_3 \leftarrow R_3 - 3R_2 \quad (\ell_{32} = 3) \quad \text{produces} \quad \begin{bmatrix} 1 & 0 & 3 \\ 0 & 2 & -4 \\ 0 & 0 & 7 \end{bmatrix} = U.$$

The unit lower triangular matrix formed from the multipliers ℓ_{ij} in the row operations is

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 3 & 1 \end{bmatrix}.$$

Check that $A = LU$ (there is no permutation matrix as there were no row swaps).

Example 3.2.6 Show that the following matrix A is non-singular but does not have an LU factorization

$$A = \begin{bmatrix} 1 & 1 & 3 \\ 2 & 2 & 2 \\ 3 & 6 & 4 \end{bmatrix}$$

Applying the following row operations

$$\begin{aligned} R_2 &\leftarrow R_2 - 2R_1 \quad (\ell_{21} = 2) \\ R_3 &\leftarrow R_3 - 3R_1 \quad (\ell_{31} = 3) \end{aligned} \quad \text{to obtain} \quad \begin{bmatrix} 1 & 0 & 3 \\ 0 & 0 & -4 \\ 0 & 3 & -5 \end{bmatrix}.$$

Now the algorithm must stop since the pivot element is 0

.

3.2.2 Existence and uniqueness

Definition 9 *(Principal submatrix) Let A be an $n \times n$ matrix. For $i = 1, 2, \ldots, n$, the principal submatrix of order i of A is an $i \times i$ matrix whose rows and columns are the first i rows and columns of A.*

Example 3.2.7 Let

$$A = \begin{bmatrix} 1 & 1 & 3 \\ 2 & 2 & 2 \\ 3 & 6 & 4 \end{bmatrix}$$

Then

$$A_1 = [1], \quad A_2 = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} \quad A_3 = A.$$

Theorem 3.2.1 *For a given matrix A of size $n \times n$, its LU factorization exists and is unique if and only if all principal submatrices A_i of A, for $i = 1, 2, \ldots, n-1$, are non-singular.*

Example 3.2.8 Use Theorem 3.2.1 to show that the matrix

$$A = \begin{bmatrix} 1 & 1 & 3 \\ 2 & 2 & 2 \\ 3 & 6 & 4 \end{bmatrix}$$

does not have an LU factorization.

Example 3.2.9 Show that the following matrix has an LU factorization and find this factorization.

$$A = \begin{bmatrix} 1 & 0 & 3 \\ 2 & 2 & 2 \\ 3 & 6 & -3 \end{bmatrix}$$

Note that A is singular.

3.2.3 Pivoting

- Gaussian elimination algorithm requires that all pivot elements are nonzero.

- In the case that a pivot element is zero, interchange rows to have a nonzero pivot element. This technique is called pivoting.

- Note that pivoting resulting in an LU factorization of PA (not A) where P is a permutation matrix:

$$PA = LU.$$

  Therefore, instead of solving $A\boldsymbol{x} = \boldsymbol{b}$ we solve $PA\boldsymbol{x} = P\boldsymbol{b}$, or $LU\boldsymbol{x} = P\boldsymbol{b}$.

- This means we solve the triangular systems $L\boldsymbol{y} = P\boldsymbol{b}$ and $U\boldsymbol{x} = \boldsymbol{y}$.

Example 3.2.10 Use pivoting to find the LU factorization of

$$A = \begin{bmatrix} 1 & 1 & 3 \\ 2 & 2 & 2 \\ 3 & 6 & 4 \end{bmatrix}$$

Is $LU = A$?

The MATLAB command `lu`

MATLAB

```
>> A = [1 0 3; 2 2 2; 3 6 -3];
>> [L1,U1,P] = lu(A)
L1 =
   1.0000   0        0
   0.6667   1.0000   0
   0.3333   1.0000   1.0000
U1 =
   3      6    -3
   0     -2     4
   0      0     0
P =
   0      0     1
   0      1     0
   1      0     0
>> [L2,U2] = lu(A)
L2 =
   0.3333    1.0000    1.0000
   0.6667    1.0000         0
   1.0000         0         0
U2 =
   3       6      -3
   0      -2       4
   0       0       0
```

Note that `L2` is not a unit lower triangular matrix. It is a "psychologically lower triangular matrix" (i.e. a product of lower triangular and permutation matrices) in `L1`, so that `A = L2*U2`.

The following Python script computes the LU factorization of a matrix A.

Python script

```
import pprint
import numpy
from scipy import linalg   # SciPy Linear Algebra Library

A = numpy.array([[7, 3, -1, 2],[3, 8, 1, -4],[-1, 1, 4, -1],[2, -4, -1, 6]])
P, L, U = linalg.lu(A)

print("A:")
pprint.pprint(A)

print("P:")
pprint.pprint(P)

print("L:")
pprint.pprint(L)

print("U:")
pprint.pprint(U)
```

The output from the code is given below:

Python output

```
A:
array([[ 7,  3, -1,  2],
       [ 3,  8,  1, -4],
       [-1,  1,  4, -1],
       [ 2, -4, -1,  6]])
P:
array([[ 1.,  0.,  0.,  0.],
       [ 0.,  1.,  0.,  0.],
       [ 0.,  0.,  1.,  0.],
       [ 0.,  0.,  0.,  1.]])
L:
array([[ 1.        ,  0.        ,  0.        ,  0.        ],
       [ 0.42857143,  1.        ,  0.        ,  0.        ],
       [-0.14285714,  0.21276596,  1.        ,  0.        ],
       [ 0.28571429, -0.72340426,  0.08982036,  1.        ]])
U:
array([[ 7.        ,  3.        , -1.        ,  2.        ],
       [ 0.        ,  6.71428571,  1.42857143, -4.85714286],
       [ 0.        ,  0.        ,  3.55319149,  0.31914894],
       [ 0.        ,  0.        ,  0.        ,  1.88622754]])
```

3.2.4 Instability and partial pivoting

Another reason for pivoting is that, sometimes a well-conditioned matrix can be unstable, as shown in the following example.

Consider the linear system

$$\begin{bmatrix} \mu & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

where $|\mu|$ is very small.

The exact solution is

$$\boldsymbol{x} = \frac{1}{1-\mu} \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

The condition number is

$$\kappa_\infty(A) = \frac{4}{|\mu - 1|} \approx 4,$$

so the problem is well-conditioned.

The LU factorization is

$$L = \begin{bmatrix} 1 & 0 \\ \mu^{-1} & 1 \end{bmatrix}, \quad \text{and} \quad U = \begin{bmatrix} \mu & 1 \\ 0 & 1 - \mu^{-1}. \end{bmatrix}$$

However, the MATLAB script `badpivot.m`

MATLAB script

```
mu = 1e-15;
A = [mu, 1; 1, 1];
b = [1, 0]';
L = [1, 0; 1/mu, 1];
U = [mu, 1; 0, 1-1/mu];
y = L \ b;
xstar = U \ y
x = ( 1 / ( 1 - mu ) ) * [-1, 1]'
relerr = norm(xstar-x, inf) / norm(x, inf)
```

gives the output

MATLAB output

```
Warning: Matrix is close to singular or badly
         scaled. Results may be inaccurate.
         RCOND = 1.000000e-30.
> In badpivot at 4

xstar = -1.11022302462516
         1.00000000000000

x = -1.00000000000000
     1.00000000000000

relerr = 0.11022302462516
```

Python script

```
import numpy
from scipy import linalg
mu = 1e-15

A = numpy.array([[mu, 1],[1,1]])
b = numpy.array([1,0])
L = numpy.array([[1,0],[1/mu,1]])
U = numpy.array([[mu,1],[0, 1-1/mu]])

y = linalg.solve(L,b)
xstar = linalg.solve(U,y)
print('xstar=',xstar)

x = numpy.array([-1,1])
x = (1/(1-mu))*x
print('x=',x)
relerr = linalg.norm(xstar-x,numpy.inf)/linalg.norm(x,numpy.inf)
print('Relative error=',relerr)
```

Python output

```
badpivot.py:10: LinAlgWarning:
Ill-conditioned matrix (rcond=1e-30): result may not be accurate.
  y = linalg.solve(L,b)
badpivot.py:11: LinAlgWarning:
Ill-conditioned matrix (rcond=1e-30): result may not be accurate.
  xstar = linalg.solve(U,y)
xstar= [-1.11022302  1.        ]
x= [-1.  1.]
Relative error= 0.1102230246251553
```

Thus, only the first decimal digit of `xstar(1)` is correct. This loss of accuracy is easily avoided by simply changing the order of the equations, i.e. we solve the equivalent system

$$\begin{bmatrix} 1 & 1 \\ \mu & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

The solution $\boldsymbol{x}$ and the condition number $\kappa(A)$ are unchanged, but now, the LU factorization is

$$L = \begin{bmatrix} 1 & 0 \\ \mu & 1 \end{bmatrix}, \quad \text{and} \quad U = \begin{bmatrix} 1 & 1 \\ 0 & 1-\mu. \end{bmatrix}$$

and the numerical solution `xstar` will be accurate to machine precision.

The following theorems discuss roundoff errors in the LU factorisation.

Theorem 3.2.2 *(Wilkinson 1961) Suppose the LU-factorization of a matrix A is computed in a system of floating point arithmetic. Then the approximate quantities L_* and U_* satisfy*

$$L_*U_* = A + \delta A,$$

where

$$\|\delta A\|_\infty \leq \rho_n(\|A\|_\infty + \|L_*\|_\infty\|U_*\|_\infty)u + O(u^2).$$

Here, ρ_n is a constant not larger than $3n$ and $u = \epsilon/2 \approx 10^{-16}$. Typically $\rho_n \ll n$ when n is large.

The following theorem is a direct consequence of Theorems 3.1.7 and 3.2.2.

Theorem 3.2.3 *Suppose that we solve $A\boldsymbol{x} = \boldsymbol{b}$ by finding the LU-factorisation $A = LU$ and then solving the triangular system $L\boldsymbol{y} = \boldsymbol{b}$ and $U\boldsymbol{x} = \boldsymbol{y}$. Assume that we perform all computations in a system of floating-point arithmetic. Then the computed solution $\boldsymbol{x}_*$ is the exact solution of a perturbed linear system*

$$(A + \delta A)\boldsymbol{x}_* = \boldsymbol{b} + \delta\boldsymbol{b},$$

and satisfies

$$\frac{\|\boldsymbol{x}_* - \boldsymbol{x}\|_\infty}{\|\boldsymbol{x}\|_\infty} \leq c_n\kappa_\infty(A)\left[\left(1 + \frac{\|L_*\|_\infty\|U_*\|_\infty}{\|A\|_\infty}\right)u + O(u^2) + \frac{\|\delta\boldsymbol{b}\|_\infty}{\|\boldsymbol{b}\|_\infty}\right].$$

Here c_n is a constant not larger than $5n$.

In practice, $c_n \ll n$ when n is large. Roundoff errors should not be a problem provided that the following 2 conditions hold

- The condition number $\kappa_\infty(A)$ is of moderate size.
- All entries of the computed factors L_* and U_* are of moderate size.

In the previous example, the condition number $\kappa_\infty(A)$ is about 4, but

$$\|L_*\|_\infty \approx \|L\|_\infty = 1 + |\mu^{-1}| \quad \text{and} \quad \|U_*\|_\infty \approx \|U\|_\infty = |1 - \mu^{-1}|.$$

Hence

$$\frac{\|L_*\|_\infty\|U_*\|_\infty}{\|A\|_\infty} = \frac{(1 + \mu^{-1})|1 - \mu^{-1}|}{2} \approx \frac{1}{2\mu^2},$$

which is large when $|\mu|$ is very small.

A remedy to that is `partial pivoting`.

The process performs row swaps (even in the case of nonzero pivot elements) so that a pivot element is larger in magnitude than all other entries in the same column lying below it. It guarantees that

$$|\ell_{ij}| \leq 1 \quad \text{for all} \quad 1 \leq j < i \leq n.$$

The MATLAB function `lu` performs this partial pivoting process.

We illustrate how pivoting is implemented by factorising the 4×4 matrix

$$A = \begin{bmatrix} 1 & 0 & -5 & 2 \\ 5 & 1 & 8 & 4 \\ -3 & 3 & 7 & 2 \\ 1 & -4 & 8 & 2 \end{bmatrix} \qquad \boxed{\begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix}}$$

We use the boxed column of numbers to keep track of the row swaps. To do the computations in Matlab, we start with

```
>> A = [1, 0,-5, 2; 5, 1, 8, 4; -3, 3, 7, 2; 1, -4, 8, 2]
>> saveA = A
```

At the first step, we find the entry in the first column that has the largest magnitude, in this case 5. To use this number as the pivot, we swap its row with row 1:

```
>> A([1 2],:) = A([2 1],:)
```

In this way, we obtain

$$A = \begin{bmatrix} 5 & 1 & 8 & 4 \\ 1 & 0 & -5 & 2 \\ -3 & 3 & 7 & 2 \\ 1 & -4 & 8 & 2 \end{bmatrix} \qquad \boxed{\begin{matrix} 2 \\ 1 \\ 3 \\ 4 \end{matrix}}$$

Now eliminate the $(2,1)$-entry by subtracting $1/5\times$ row 1 from row 2, but instead of storing 0 in the $(2,1)$-position, store the *multiplier* $1/5 = 0.2$. The commands

```
>> A(1,2) = A(1,2) / A(1,1)
>> A(2,2:4) = A(2,2:4)-A(2,1)*A(1,2:4)
```

give

$$A = \begin{bmatrix} 5.0 & 1.0 & 8.0 & 4.0 \\ 0.2 & -0.2 & -6.6 & 1.2 \\ -3.0 & 3.0 & 7.0 & 2.0 \\ 1.0 & -4.0 & 8.0 & 2.0 \end{bmatrix} \qquad \boxed{\begin{matrix} 2 \\ 1 \\ 3 \\ 4 \end{matrix}}$$

To elminate the $(3,1)-$ and $(4,1)-$entries (and store the multipliers), we do

```
>> A(3,1) = A(3,1)/A(1,1)
>> A(3,2:4) = A(3,2:4) - A(3,1)*A(1,2:4)
>> A(4,1) = A(4,1)/A(1,1)
>> A(4,2:4) = A(4,2:4) - A(4,1)*A(1,2:4)
```

and obtain

$$A = \left[\begin{array}{cccc|c} 5.0 & 1.0 & 8.0 & 4.0 & 2 \\ 0.2 & -0.2 & -6.6 & 1.2 & 1 \\ -0.6 & 3.6 & 11.8 & 4.4 & 3 \\ 0.2 & -4.2 & 6.4 & 1.2 & 4 \end{array}\right]$$

Looking at the entries in positions 2 to 4 of the second column, we see that the largest in magnitude is -4.2, so we make this the next pivot by swapping rows 2 and 4.

```
>> A([2 4],:) = A([4 2],:)
```

In this way, we have

$$A = \left[\begin{array}{cccc|c} 5.0 & 1.0 & 8.0 & 4.0 & 2 \\ 0.2 & -4.2 & 6.4 & 1.2 & 4 \\ -0.6 & 3.6 & 11.8 & 4.4 & 3 \\ 0.2 & -0.2 & -6.6 & 1.2 & 1 \end{array}\right]$$

Now eliminate the $(3,2)-$ and $(4,2)-$entries, storing the multipliers:

```
>> A(3,2) = A(3,2)/A(2,2)
>> A(3,3:4) = A(3,3:4) - A(3,2)*A(2,3:4)
>> A(4,2) = A(4,2)/A(2,2)
>> A(4,3:4) = A(4,3:4) - A(4,2)*A(2,3:4)
```

and obtain

$$A = \left[\begin{array}{cccc|c} 5.0000 & 1.0000 & 8.0000 & 4.0000 & 2 \\ 0.2000 & -4.2000 & 6.4000 & 1.2000 & 4 \\ -0.6000 & -0.8571 & 17.2857 & 5.4286 & 3 \\ 0.2000 & 0.0476 & -6.9048 & 1.1429 & 1 \end{array}\right]$$

Since the $(3,3)-$entry is larger in magnitude than the $(4,3)-$entry, no row swap is needed and we do

```
>> A(4,3) = A(4,3) / A(3,3)
>> A(4,4) = A(4,4) - A(4,3) * A(3,4)
```

to complete the factorization:

$$A = \begin{bmatrix} 5.0000 & 1.0000 & 8.0000 & 4.0000 \\ 0.2000 & -4.2000 & 6.4000 & 1.2000 \\ -0.6000 & -0.8571 & 17.2857 & 5.4286 \\ 0.2000 & 0.0476 & -0.3994 & 3.3113 \end{bmatrix} \quad \boxed{\begin{matrix} 2 \\ 4 \\ 3 \\ 1 \end{matrix}}$$

Indeed, the lower triangular factor is

$$L = \begin{bmatrix} 1.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.2000 & 1.0000 & 0.0000 & 0.0000 \\ -0.6000 & -0.8571 & 1.0000 & 0.0000 \\ 0.2000 & 0.0476 & -0.3994 & 1.0000 \end{bmatrix},$$

and the upper triangular factor is

$$U = \begin{bmatrix} 5.0000 & 1.0000 & 8.0000 & 4.0000 \\ 0.0000 & -4.2000 & 6.4000 & 1.2000 \\ 0.0000 & 0.0000 & 17.2857 & 5.4286 \\ 0.0000 & 0.0000 & 0.0000 & 3.3113 \end{bmatrix}.$$

If we define the permutation matrix

$$P = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}.$$

then

$$PA = LU.$$

To check this, do

```
>> L = tril(A,-1) + eye(4,4), U = triu(A), A = saveA
>> P = eye(4,4), P = P([2 4 3 1],:)
>> norm(L*U-P*A,inf)
ans = 8.8818e-16
```

To solve $A\boldsymbol{x} = \boldsymbol{b}$, we solve the triangular systems $L\boldsymbol{y} = P\boldsymbol{b}$ and $U\boldsymbol{x} = \boldsymbol{y}$, so that

$$PA\boldsymbol{x} = LU\boldsymbol{x} = L\boldsymbol{y} = P\boldsymbol{b}.$$

3.2.5 Computational cost

Definition 10 *(Flop) A flop is a floating-point operation, i.e., one of $a + b$ or ab where a and b are two floating point numbers.*

Example 3.2.11 To compute an inner product

$$\boldsymbol{x} \cdot \boldsymbol{y} = \sum_{i=1} x_i \cdot y_i$$

we use $2n$ flops.

Counting flops in this way gives some indication of the computational cost of an algorithm.

Example 3.2.12 The cost of LU factorization of an $n \times n$ matrix is roughly $\frac{2n^3}{3}$ flops.

We recall that the LU factorization is derived from the Gaussian elimination procedure. Consider the first step of the Gaussian elimination procedure of a matrix A, given by

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}$$

The transformed matrix is

$$\widetilde{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ 0 & \widetilde{a_{22}} & \dots & \widetilde{a_{2n}} \\ \vdots & \vdots & \vdots & \vdots \\ 0 & \widetilde{a_{n2}} & \dots & \widetilde{a_{nn}} \end{bmatrix},$$

where

$$\begin{aligned} \widetilde{a_{2j}} &= a_{2j} - k_2 a_{1j}, \quad j = 2, \dots, n \\ \widetilde{a_{3j}} &= a_{3j} - k_3 a_{1j}, \quad j = 2, \dots, n \\ \vdots &= \vdots \\ \widetilde{a_{nj}} &= a_{nj} - k_n a_{1j}, \quad j = 2, \dots, n \end{aligned}$$

The entries $a_{21}, \dots, a_{n1}$ will be converted to zeros, so first we need $(n-1)$ divisions to compute the elimination factors $k_2 = a_{21}/a_{11}, \dots, k_n = a_{n1}/a_{11}$.

To transform the $(n-1) \times (n-1)$ sub-matrix starting from the second column and the second row, we need $(n-1)^2$ subtractions and $(n-1)^2$ multiplications.

Then we need to move to the 2nd column, and to compute $(n-2)$ factors $\widetilde{a_{32}}/\widetilde{a_{22}}, \dots, \widetilde{a_{n2}}/\widetilde{a_{22}}$ using $(n-2)$ divisions.

To transform the $(n-2) \times (n-2)$ sub-matrix starting from the second column and the second row, we need $(n-2)^2$ subtractions and $(n-2)^2$ multiplications and so on.

We need to repeat the process until all entries below the diagonal become zeros. The total number of divisions is

$$(n-1)+(n-2)+(n-3)+\ldots+1=\frac{n(n-1)}{2}$$

The total number of subtractions is

$$(n-1)^2+(n-2)^2+\ldots+1=\sum_{i=1}^{(n-1)} i^2=\frac{n^3}{3}-\frac{n^2}{2}+\frac{n}{6}$$

The total number of multiplications is

$$(n-1)^2+(n-2)^2+\ldots+1=\sum_{i=1}^{(n-1)} i^2=\frac{n^3}{3}-\frac{n^2}{2}+\frac{n}{6}$$

The total number of floating operations for LU factorization is

$$\frac{2n^3}{3}-n^2+\frac{n}{3}+\frac{n(n-1)}{2}=O(n^3).$$

Example 3.2.13 Counting the flops of solving the lower triangular system $L\boldsymbol{x}=\boldsymbol{b}$.

The lower triangular linear system is

$$\begin{array}{lllll}
\ell_{11}x_1 & & & & =b_1\\
\ell_{21}x_1 & +\ell_{22}x_2 & & & =b_2\\
\ell_{31}x_1 & +\ell_{32}x_2 & +\ell_{33}x_3 & & =b_3\\
\vdots & & & & =\vdots\\
\ell_{n1}x_1 & +\ell_{n2}x_2 & +\ell_{n3}x_3 & \ldots\quad +\ell_{nn}x_n & =b_n
\end{array}$$

To compute x_1 by $x_1=b_1/\ell_{11}$, we need 1 division. To compute x_2 by $x_2=(b_2-\ell_{21}x_1)/\ell_{22}$ we need 1 multiplication, 1 subtraction and 1 division. To compute x_3 by $x_3=(b_3-\ell_{31}x_1-\ell_{32}x_2)/\ell_{33}$ we need 2 multiplications, 2 subtractions and 1 division. ... To compute $x_n=(b_n-\ell_{n1}x_1-\ldots-\ell_{n,n-1}x_{n-1})/_{\ell_{nn}}$ we need $(n-1)$ multiplications, $(n-1)$ subtractions and 1 division. So in total we need n divisions, and

$$1+2+3+\ldots(n-1)=\frac{n(n-1)}{2}\text{ multiplications}$$

and

$$1+2+3+\ldots(n-1)=\frac{n(n-1)}{2}\text{ subtractions.}$$

Total number of flops is therefore

$$n+\frac{n(n-1)}{2}+\frac{n(n-1)}{2}=n+n(n-1)=n^2.$$

Example 3.2.14 Counting the flops of solving the linear system $A\boldsymbol{x} = \boldsymbol{b}$ using the LU factorisation algorithm.

- Computing L and U uses roughly $\frac{2n^3}{3}$ flops. We say that the complexity in finding the LU factorization is $O(n^3)$.

- Once we have found L and U, solving $L\boldsymbol{y} = \boldsymbol{b}$ and $U\boldsymbol{x} = \boldsymbol{y}$ uses only roughly $2n^2$ flops. Thus the complexity is $O(n^2)$.

We conclude that, when we solve a linear system by LU factorization, most of the time is spent to find the LU factorization. Therefore, when the same system is solved with different right hand sides, it is recommended that the factorization is carried once and stored for later use.

The following MATLAB script `timelu.m` estimates the exponent p such that the CPU time compute the LU factorization of a matrix A is approximately proportional to n^p.

MATLAB script

```
steps = 10;
neqns = 500 * [1:steps];
tlu = zeros(1,steps);
fprintf('Timing LU factorization of an n-by-n matrix.\n\n')
fprintf(' n seconds exponent\n\n')
for k = 1:steps
   n = neqns(k);
   A = eye(n) + 0.5 * rand(n);
   start = cputime;
   [L,U] = lu(A);
   finish = cputime;
   tlu(k) = finish - start;
   if k == 1 | tlu(k-1) < eps
       fprintf('%8d %12.4f\n', n, tlu(k))
   else
       p = log( tlu(k)/tlu(k-1) ) / log( neqns(k)/neqns(k-1));
       fprintf('%8d %12.4f %10.2f\n', n, tlu(k), p)
   end
end
```

On my desktop (Intel i7 quad core), I got the following output

MATLAB output

```
Timing LU factorization of an n-by-n matrix.

      n        seconds     exponent

    500        0.1100
   1000        0.1100        0.00
   1500        0.3100        2.56
   2000        0.6600        2.63
   2500        1.2300        2.79
   3000        2.1100        2.96
   3500        3.1900        2.68
   4000        4.5700        2.69
   4500        6.4100        2.87
   5000        8.9200        3.14
```

The equivalent Python script `timelu.py` is listed below.

Python script

```
import numpy as np
import scipy
import scipy.linalg
import time
# main program

steps = 10
tlu = np.zeros(steps) # a matrix with 'steps' zeros
print('Timing LU factorization of a n-by-n matrix\n')
print('   n   seconds    exponent \n')
for k in range(0,steps):
    n = (k+1)*1000  # matrix sizes increased by a factor of 500
    # A = I + 0.5*B, where B is a random matrix
    A = np.identity(n) + 0.5*np.random.rand(n,n)
    start = time.time()
    P, L, U = scipy.linalg.lu(A)
    finish = time.time()
    tlu[k] = finish - start
    if (k == 0):
        print("%5d" % n,"%.6f" % tlu[k])
    else:
        p = np.log( tlu[k]/tlu[k-1] )/ np.log( (k+1)/k)
        print("%5d" % n, "%2.6f" % tlu[k], "%.2f" % p)
```

3.3 Cholesky factorisation

3.3.1 Symmetric and positive definite matrices

Definition 11 *(Symmetric) A matrix $A = [a_{ij}] \in \mathbb{R}^{n\times n}$ is symmetric if $A^T = A$, i.e., if*

$$a_{ji} = a_{ij} \quad \textit{for all} \quad i \textit{ and } j.$$

Definition 12 *(Positive definiteness) The matrix $A \in \mathbb{R}^{n\times n}$ is said to be positive-definite if*

$$\boldsymbol{x}^T A\boldsymbol{x} = \boldsymbol{x}\cdot(A\boldsymbol{x}) > 0 \textit{ for all } \boldsymbol{x} \in \mathbb{R}^n \textit{ except } \boldsymbol{x} = 0,$$

or in other words if

$$\sum_{i=1}^{n}\sum_{j=1}^{n} x_i a_{ij} x_j > 0 \textit{ except when } x_1 = x_2 = \ldots = x_n = 0.$$

Example 3.3.1 The matrix

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

is positive definite. In fact,

$$\begin{aligned} \boldsymbol{x}^T A\boldsymbol{x} &= 2x_1^2 - 2x_1x_2 + 2x_2^2 - 2x_2x_3 + 2x_3^2 \\ &= x_1^2 + (x_1 - x_2)^2 + (x_2 - x_3)^2 + x_3^2 \\ &> 0, \end{aligned}$$

if $\boldsymbol{x} = (x_1, x_2, x_3) \neq 0$.

Theorem 3.3.1 *(A criterion for positive definiteness) Let $A \in \mathbb{R}^{n\times n}$ be a symmetric matrix. Then A is positive definite if and only if*

$$det(A_k) > 0 \text{ for all } k = 1, \ldots, n,$$

where A_k is the principal matrix of order k of A.

Suppose $A = [a_{ij}]_{i,j=1}^n$,i.e.

$$A = \begin{bmatrix} a_{11} & a_{12} & \ldots & a_{1n} \\ \vdots & \vdots & \ldots & \vdots \\ a_{n1} & a_{n2} & \ldots & a_{nn} \end{bmatrix},$$

the principle matrix of order k of A is defined by

$$A_k = \begin{bmatrix} a_{11} & a_{12} & \ldots & a_{1k} \\ \vdots & \vdots & \ldots & \vdots \\ a_{k1} & a_{k2} & \ldots & a_{kk} \end{bmatrix}.$$

Proof. The theorem can be proved by induction on n. A proof is given in "Linear Algebra" by Kenneth Hoffman and Ray Kunze, page 249–251. Here we just sketch the proof.

Suppose the result is true for all $(n-1)\times(n-1)$ matrices. Using elementary row operations, we can reduce A to A', where

$$A' = \begin{bmatrix} a_{11} & 0 & 0 & \ldots & 0 \\ 0 & a'_{22} & a'_{23} & \ldots & a'_{2n} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & a'_{n2} & a'_{n3} & \ldots & a'_{nn} \end{bmatrix} =: \begin{bmatrix} a_{11} & \mathbf{0} \\ \mathbf{0} & B \end{bmatrix},$$

where B is the $(n-1)\times(n-1)$ lower sub-matrix. We can show that all the principal matrix of B have positive determinants since $a_{11} > 0$ and

$$\det(B_k) = a_{11}\det(A_{k+1}) > 0, \qquad k = 1, \ldots, n-1.$$

By the induction hypothesis, B is positive definite. Hence A' is positive definite because

$$\boldsymbol{x}^T A'\boldsymbol{x} = a_{11}|x_1|^2 + \sum_{k=2}^{n}\sum_{j=2}^{n} a'_{kj}x_j x_k > 0.$$

Since A' is obtained from A through elementary row operations, it follows that A is positive definite. □

Theorem 3.3.2 *Let $A \in \mathbb{R}^{n\times n}$ be a symmetric matrix. If all the eigenvalues of A are positive then A is positive definite.*

Proof. Since A is symmetric, there are an diagonal matrix D and an orthogonal matrix P so that $A = P^T DP$ and $PP^T = I$. In fact, the diagonal entries of D are the eigenvalues λ_j (for $j = 1, \ldots, n$) of A. Hence, with $\boldsymbol{w} := P\boldsymbol{x}$,

$$\boldsymbol{x}^T A\boldsymbol{x} = \boldsymbol{x}^T P^T DP\boldsymbol{x} = \boldsymbol{w}^T D\boldsymbol{w} = \sum_{j=1}^{n} \lambda_j w_j^2.$$

So if all the eigenvalues of A are positive then $\boldsymbol{x}^T A\boldsymbol{x} > 0$, and the matrix A is positive definite. □

Example 3.3.2 Consider again the symmetric matrix in the previous example. Then

$$A_1 = [2], \quad A_2 = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \quad A_3 = A.$$

Since

$$\det(A_1) = 2 > 0, \quad \det(A_2) = 3 > 0, \quad \det(A_3) = 4 > 0,$$

A is positive definite. The eigenvalues of A can be found using the following MATLAB commands

MATLAB commands

```
>> A = [2 -1 0; -1 2 -1; 0 -1 2];
>> eig(A)

ans =

    0.5858
    2.0000
    3.4142
```

Python script

```
import numpy as np
from numpy.linalg import eig
A = np.array([[2, -1, 0],
              [-1, 2,-1],
              [0, -1, 2]])
w,v = eig(A)
print('Eigenvalue:', w)
print('Eigenvector', v)
```

Python output

```
Eigenvalue: [3.41421356 2.         0.58578644]
Eigenvector
[[-5.00000000e-01 -7.07106781e-01  5.00000000e-01]
 [ 7.07106781e-01  4.05405432e-16  7.07106781e-01]
 [-5.00000000e-01  7.07106781e-01  5.00000000e-01]]
```

All the eigenvalues are positive. Hence A is positive definite.

3.3.2 Existence and uniqueness

Theorem 3.3.3 *A matrix $A \in \mathbb{R}^{n\times n}$ is symmetric and positive-definite if and only if A has a Cholesky factorization, i.e.,*

$$A = R^T R$$

where

1. *R is upper triangular, i.e., $r_{ij} = 0$ for $1 \le j < i \le n$;*
2. *$r_{ii} > 0$ for $i = 1, 2, \ldots, n$.*

Proof. Suppose $A = R^T R$, then

$$\boldsymbol{x}^T A\boldsymbol{x} = \boldsymbol{x}^T R^T R\boldsymbol{x} = \|R\boldsymbol{x}\|_2^2 \ge 0.$$

If $R\boldsymbol{x} = 0$ then by using the conditions 1. and 2, we deduce that $\boldsymbol{x} = \boldsymbol{0}$. Hence the matrix A is positive definite.

The existence of the Cholesky factorisation of a symmetric positive definite matrix can be found in a book by Golub and Van Loan (1996). □

Example 3.3.3 If

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

then $A = R^T R$ where

$$R = \begin{bmatrix} \sqrt{2} & -1/\sqrt{2} & 0 \\ 0 & \sqrt{3/2} & -\sqrt{2/3} \\ 0 & 0 & 2/\sqrt{3} \end{bmatrix}.$$

The MATLAB built-in function `chol` produces this factorization

MATLAB commands

```
>> A = [2 -1 0; -1 2 -1; 0 -1 2];
>> R = chol(A)
```

Python script

```
import pprint
import numpy as np
from scipy import linalg
A = np.array([[2, -1, 0],
              [-1, 2,-1],
              [0, -1, 2]])
R = linalg.cholesky(A,lower=False)
print('R= ')
pprint.pprint(R)
```

Python output

```
R=
array([[ 1.41421356, -0.70710678,  0.        ],
       [ 0.        ,  1.22474487, -0.81649658],
       [ 0.        ,  0.        ,  1.15470054]])
```

3.3.3 Computational cost

The number of flops to compute R is about $n^3/3$, i.e., half the cost of an LU-factorization.

Once R is found, we can solve

$$A\boldsymbol{x} = \boldsymbol{b}$$

at a cost of about n^2 flops by solving the triangular systems $R^T\boldsymbol{y} = \boldsymbol{b}$ and $R\boldsymbol{x} = \boldsymbol{y}$, so that

$$A\boldsymbol{x} = R^T R\boldsymbol{x} = R^T\boldsymbol{y} = \boldsymbol{b}.$$

3.3.4 Stability

Wilkinson showed that the relative rounding error in the computed solution satisfies

$$\frac{\|\boldsymbol{x}_* - \boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2} \leq \rho_n \kappa_2(A)\epsilon,$$

where ρ_n is given in Theorem 3.2.2.